

Caching

02 October 2026 11:25 AM

Reading from a database or calling another service takes time. Under heavy traffic, that cost adds up quickly. A **cache** is a fast storage layer that keeps copies of frequently used data. When the application needs that data again, it can read from the cache instead of going all the way back to a slower system. This chapter breaks down caching: how it works, why systems use it, how to measure it, and when to add a cache in a system.

1. What is Caching?

Caching = storing frequently used data in a faster location so we can reuse it later.

Suppose our application needs:

Product 123

Without caching:

```
User
↓
Application
↓
Database
↓
Product 123
↓
User
```

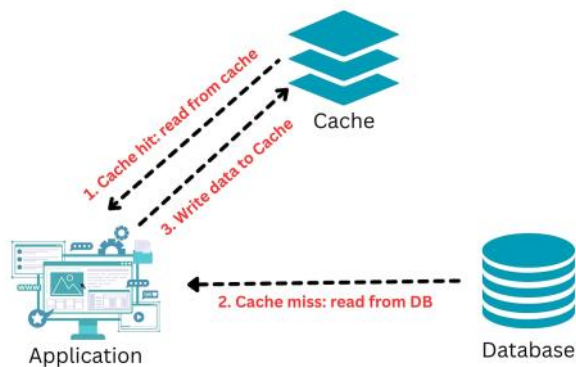
If 10,000 users request the same product, we might hit the database 10,000 times.

With caching:

```

      |
      | Cache HIT → Return data
      |
User → Application → Cache
      |
      | Cache MISS
      |
      | Database
      |
      | Store in Cache
      |
      | Return data
      |
```

The first request may be a little slower because the data isn't cached yet. But subsequent requests can be served from the cache.



2. Cache Hit vs Cache Miss

This is the **most important basic concept**.

Cache Hit

The requested data **already exists in the cache**.

```
Request
↓
Check Cache
↓
Data Found?
↓ YES
Return Cached Data
So:
```

Cache Hit = Data found in cache

No database call is required.

Cache Miss

The requested data is **not in the cache**.

Request



Check Cache



Data Found?



Database



Get Data



Store in Cache



Return Data

So:

Cache Miss = Data not found → go to database

The source describes exactly these two paths.

Remember

HIT → Cache has data → Return immediately

MISS → Cache doesn't have data → DB → Cache → Return

3. Why Do We Use Caching?

Caching mainly improves **performance**.

There are three major benefits:

Benefit	Meaning
Lower Latency	Response comes faster
Reduced Load	Fewer requests reach DB/services
Higher Throughput	Same infrastructure handles more requests

3.1 Lower Latency

Cache is usually much faster than going to a database or another service, especially when the cached data is in memory.

Database → relatively expensive



Cache → faster



Memory → very fast

So:

Cache hit = faster response

3.2 Reduced Database Load

Imagine:

1000 requests



900 cache hits

100 cache misses



Only 100 requests reach DB

The database doesn't need to process all 1000 requests.

3.3 Higher Throughput

Because the database is doing less work, the overall system can process more requests using the same infrastructure.

So caching doesn't only make **one request faster**.

It also helps the **whole system scale**.

4. Cache Hit Ratio

The **cache hit ratio** tells us what percentage of requests are served directly from the cache.

Formula

$$\text{Cache Hit Ratio} = \frac{\text{Cache Hits}}{\text{Total Requests}} \times 100$$

Example

Suppose:

Total requests = 1,000,000

Cache hits = 900,000

Then:

$$\frac{900000}{1000000} \times 100 = 90\%$$

So the **cache hit ratio = 90%**.

That means 90% of requests were handled by the cache.

Very Important at Scale

Suppose:

10 million requests/second

95% cache hit ratio

Sounds excellent.

But:

5% misses

= $5\% \times 10,000,000$

= 500,000 requests/second

So **500,000 requests/sec still reach the backend**.

This gives you an important System Design rule:

Size your backend/database for cache misses, not total traffic.

5. Caching Layers in a System

Caching doesn't necessarily happen at only one place.

A large system can have **multiple caching layers**.

Think of it like this:

User



Browser Cache



CDN Cache



Application Cache



Distributed Cache



Database



Database Internal Cache

Each layer has a different purpose.

5.1 Browser Cache

The browser can cache:

- Images
- CSS
- JavaScript
- API responses

It is the cache **closest to the user**.

If the browser already has a valid copy:

User



Browser Cache



Data found



No network request needed

HTTP headers such as Cache-Control and ETag help control this behavior.

5.2 CDN Cache

CDN = **Content Delivery Network**

A CDN stores content at locations closer to users around the world.

Example:



Instead of every user contacting your main server, they can get cached content from a nearby CDN location.

Best for:

- Images
- Videos
- Static files
- Public pages
- Media

5.3 Application Cache

The application itself can store frequently used data in memory.

Application Server



Local Memory



Cached Data

Very fast because the data is already inside the application process.

Problem

Suppose we have:

Server 1 → Cache A

Server 2 → Cache B

Server 3 → Cache C

Each server has its **own cache**.

So:

- Memory gets duplicated.
- Different servers can have different cached values.

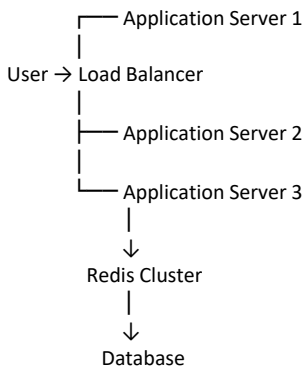
5.4 Distributed Cache

Instead of every application server having its own cache, multiple servers can share one distributed cache.

Common examples:

- Redis
- Memcached

Architecture:



Advantages:

- Multiple application servers share the same cache.
- More cache capacity.
- Servers can see the same cached data.

Trade-off:

Distributed cache requires a network call, so it is slower than local application memory.

5.5 Database Cache

Even the database itself can maintain internal caches.

Frequently accessed database pages can stay in memory rather than being repeatedly read from disk.

Application



Database



Database Cache



Disk

The application usually doesn't manage this cache directly; the database handles it internally.

6. What Data Should We Cache?

Not everything should be cached.

Caching works best when:

1. Data is requested **frequently**
2. Retrieving/calculating it is **expensive**
3. The same result can be reused

Good examples:

- Frequently viewed product details
- Expensive database queries
- API responses
- Configuration data
- User sessions
- Results of complex computations

Think:

Frequently requested

+

Expensive to generate

+

Can tolerate some staleness

↓

GOOD CACHE CANDIDATE

7. What Should We NOT Cache?

Some data is a poor fit.

Data that changes constantly

If data changes every second:

DB → value A

Cache → value A

DB → value B

Cache → old value A

The cache quickly becomes stale.

Data that must always be fresh

Example:

Bank account balance

Serving an outdated balance could be unacceptable.

Huge data that is rarely accessed

Suppose:

1 GB object

requested once every 5 hours

Keeping it in cache may waste memory.

The source highlights these three poor-fit cases.

8. Four Questions Before Caching

Before adding something to cache, ask:

1. How expensive is it to retrieve?
2. How often is it requested?
3. How frequently does it change?
4. How much staleness can we tolerate?

These questions are very useful in an HLD interview.

9. Cache Keys

Every cached value needs an identifier.

That identifier is called a **cache key**.

For example:

product:123 → Product data

user:456 → User data

search:shoes:page:2 → Search results

Think:

CACHE

Key → Value

product:123	→ Product
user:456	→ User
session:789	→ Session

10. Good vs Bad Cache Keys

Problem 1: Key Too Broad

Suppose:

cart → User 1's cart

Then User 2 also requests:

cart

They could get User 1's data.

✗ Bad.

Instead:

cart:user:101

cart:user:102

Problem 2: Key Too Specific

Suppose:

product:123:10:01

product:123:10:02

product:123:10:03

You are storing the same product repeatedly.

This can produce many cache misses and reduce cache effectiveness.

The source describes both the **too broad** and **too specific** problems.

Important Rule

A cache key should contain every parameter that changes the result.

For example:

search:shoes:sort=price:page=2

is different from:

search:shoes:sort=rating:page=2

because the returned data is different.

11. Namespaces and Versioning

It is common to organize keys using namespaces:

user:123

product:456

session:789

You can also use versions:

v1:product:123

v2:product:123

Suppose the product response format changes.

Instead of deleting every old cache entry individually:

Application starts using:

v2:product:123

Now the old v1 entries are no longer used.

The source describes versioning as a way to invalidate groups of entries when the format changes.

12. TTL — Time To Live

A cache entry shouldn't necessarily live forever.

TTL = Time To Live

It defines how long the cached value remains valid.

Example:

product:123

TTL = 5 minutes

Flow:

t = 0

Request

↓

Cache MISS

↓
 Database
 ↓
 Store in cache
 TTL = 5 min

t = 2 min
 Request
 ↓
 Cache HIT
 ↓
 Return cached data

t = 6 min
 Request
 ↓
 TTL expired
 ↓
 Cache MISS
 ↓
 Database
 ↓
 Store fresh data

13. Short TTL vs Long TTL

There is a trade-off.

Short TTL

Short TTL
 ↓
 Data expires quickly
 ↓
 Fresher data
 ↓
 More cache misses
 ↓
 More DB traffic

Long TTL

Long TTL
 ↓
 Data stays cached longer
 ↓
 Higher hit ratio
 ↓
 Less DB traffic
 ↓
 Greater chance of stale data

So there is **no universal perfect TTL**.

It depends on:

How quickly does the data change, and how much stale data can the application tolerate?

Example from the source:

Data	Typical TTL
Static configuration	Hours
Product data	Few minutes
Highly dynamic data	Very short / another invalidation strategy

14. When Should We Add a Cache?

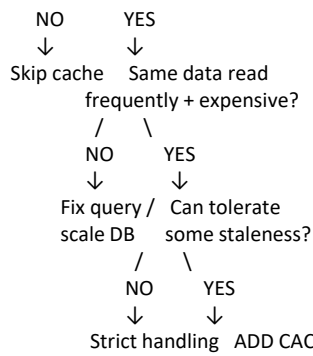
Don't automatically add caching to every system.

A cache itself introduces complexity:

- Expiration
- Invalidation
- Stale data
- Failures
- Consistency problems

Use this decision flow:

Performance problem?
 / \



The source gives essentially this decision process.

15. Three Signs That Caching Will Help

Remember these three:

1. High read frequency

The same data is requested many times.

2. Expensive read/computation

Getting the data requires significant DB work or computation.

3. Repeated access

The same result can be reused.

High read frequency
+
Expensive operation
+
Same data repeatedly
↓
CACHE